Procedural Content Generation using Perlin (among others)

Computing Club (03/03/2026)

Erik Fredericks, frederer@gvsu.edu

Winter 2026

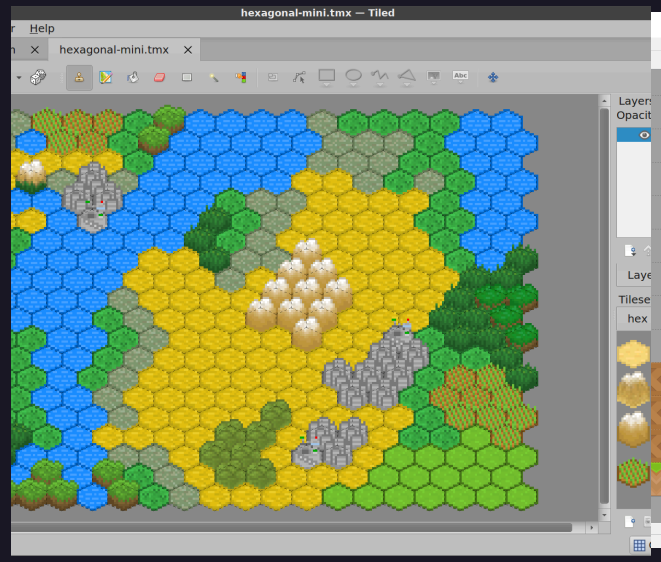
https://efredericks.github.io

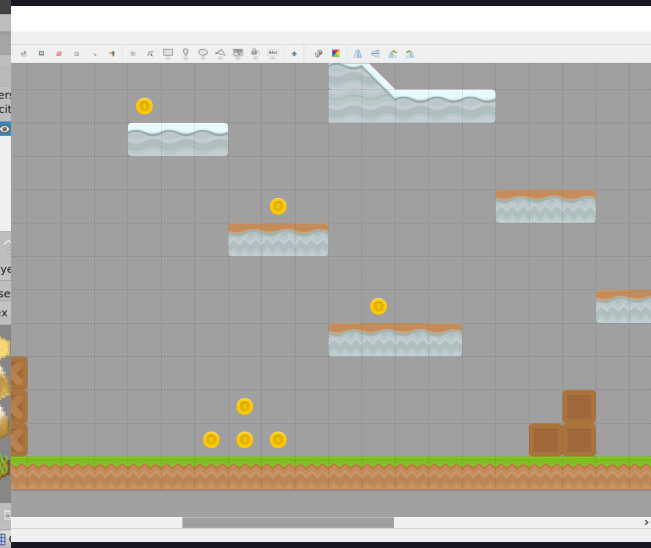
(Background image: Etienne Jacob)

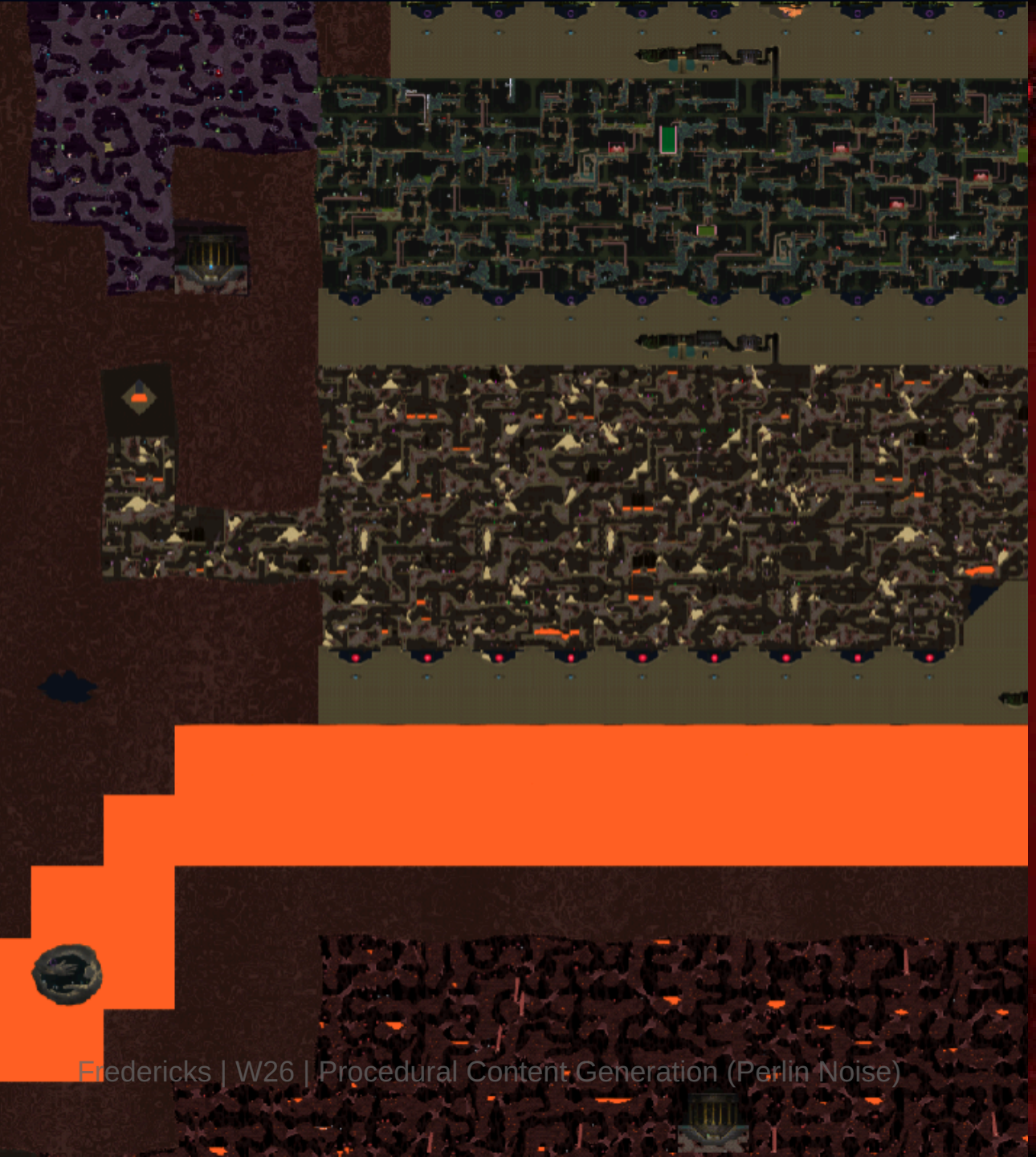
But first, how do we make a game map?

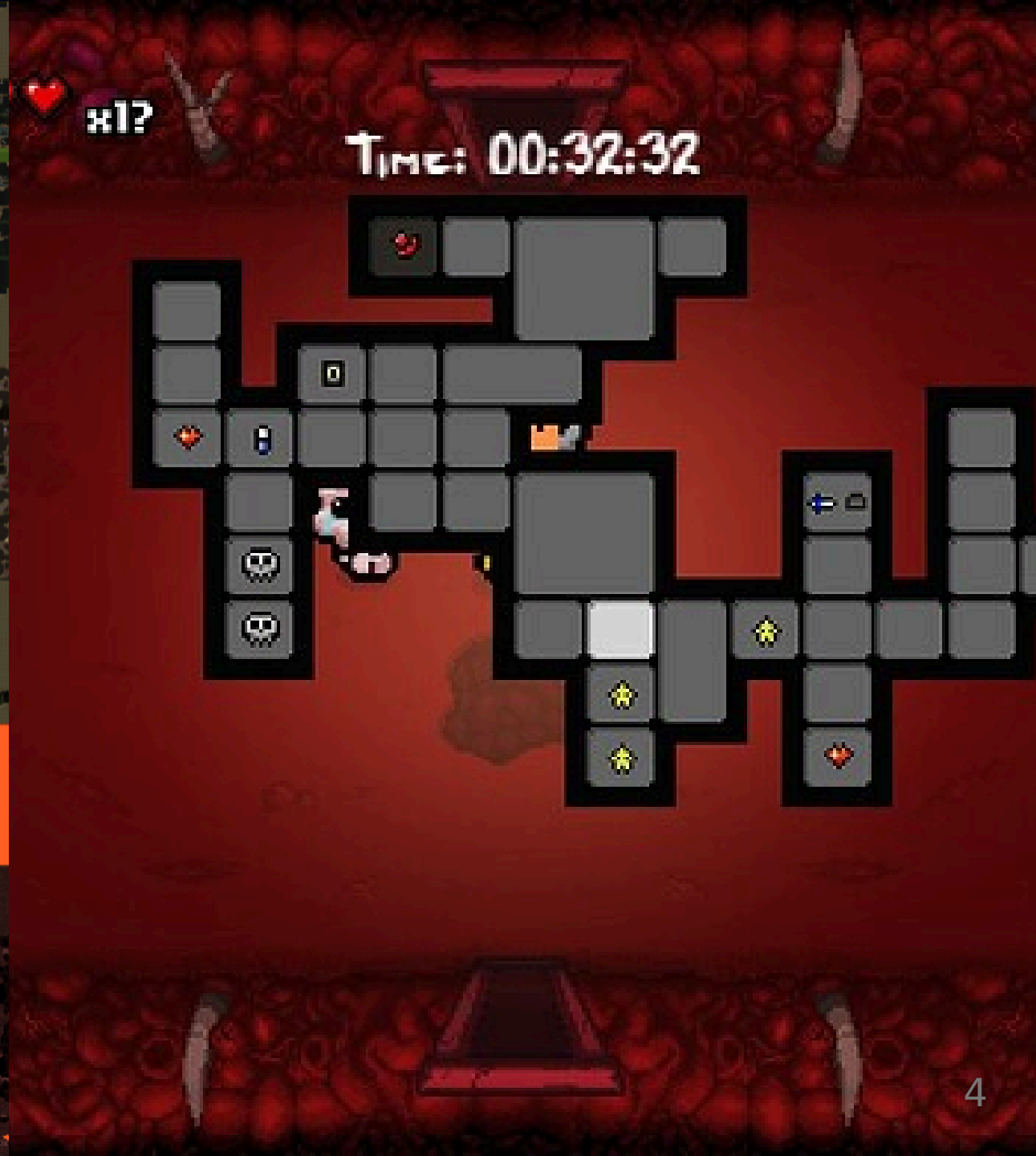
(and, what does the data structure look like?)

Any reason for not doing it by hand?








x17
Time: 00:32:32

Procedural Content Generation (PCG)

What is PCG?

- Algorithmically-placing content!

Why is PCG?

- Save the developer/designer time and effort!
  - One would think...

Why bother talking about this?

- I recently sent out a paper on it and thought you might find it interesting
- Plus, useful for gamejam things

Basic concept

Use math/algorithms to place content intelligently

- Dungeons, items, etc.

Noise functions:

- Calculate a noise value based on inputs and configurations
- Your job is to map that value to something useful

For example - Perlin noise

Typically, returns a value in [-1.0, 1.0], though p5js uses [0.0, 1.0]

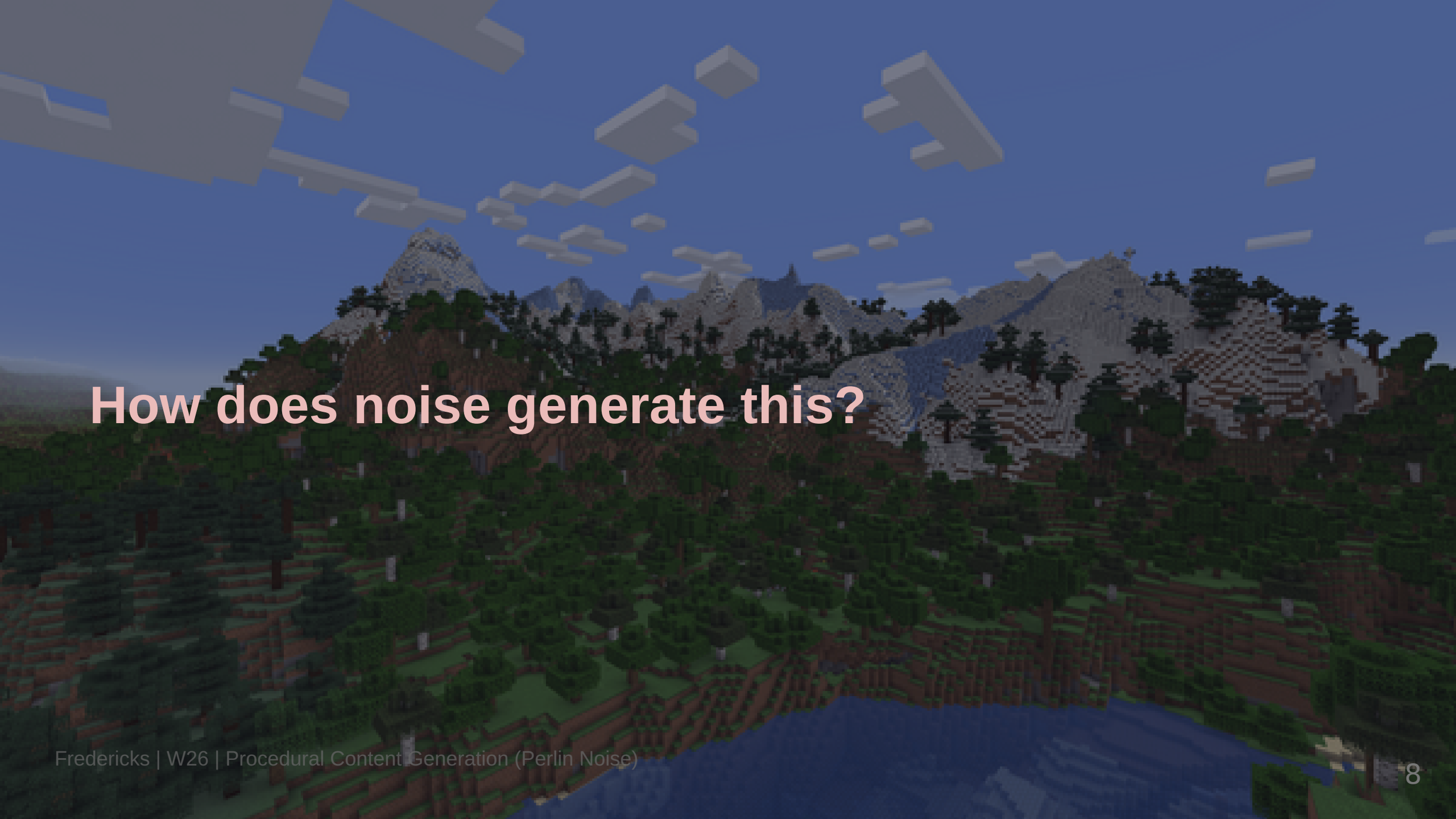
For example (in p5):

```
let n = noise(x * 0.01, y * 0.01);
```

- `n = [0.0, 0.5] -- (x, y) -> water`
- `n = (0.5, 0.6] -- (x, y) -> beach`
- `n = (0.6, 0.9] -- (x, y) -> grass`
- `n = (0.9, 1.0] -- (x, y) -> rock`

You do *effectively* the same in any language/editor!

There are other noise functions! Worley noise, Simplex noise, etc.



How does noise generate this?

https://tinyurl.com/24c9sftw

Demo time

This time I made a pre-baked p5js template for you. Go to this link and `File -> Duplicate`

- Make sure you login and save often

The sketch

- You should have a little ASCII happy face that you can move around with your arrow keys
- Your game map is represented as a **two-dimensional grid** where the first index is the row and the second index is the column
- There is also a basic camera that follows your player so that we can make big maps

The map

```
[['#', '#', '#', ..., '#'],
 ['#', '', '', ..., '#'],
 ['#', '.', 't', ..., '#'],
 [...],
 ['#', '#', '#', ..., '#']]
```

`game_map[2][1]` **returns a ???**

- `.`

By default, the map puts walls on the outside and nothing on the inside - your job is to fill it with things

Try hitting `~` and see what happens!

Valid things:

Character	Walkable?	Represents
`#`	No	Stone wall
`t`	No	Tree
`w`	No	Water
` `	Yes	Empty space
`.`	Yes	Pebbles
`g`	Yes	Grass

Try setting some random cells to these characters!

- such as, `game_map[2][3] = 'g';` in `setup()` AFTER the game map is created

Is this something interesting?

First, let's randomize things

After the call to `setupGameMap()` (which again, just gives us an empty grid with borders)

```
for (let r = 1; r < num_rows - 1; r++) {
  for (let c = 1; c < num_cols - 1; c++) {
    let r = random();
    let ch = '';
    if (r > 0.8) ch = '#';

    game_map[r][c] = ch;
  }
}
```

Now let's give it a bit of detail

First, comment out what you did inside that double loop.

```
const zoom = 0.01;
for (let r = 1; r < num_rows - 1; r++) {
  for (let c = 1; c < num_cols - 1; c++) {
    let ch = '';
    let n = noise(c * zoom, r * zoom);


    // fill in with code from next slide


    game_map[r][c] = ch;
  }
}
```

The noise bit

```
if (n <= 0.5) ch = 'w';
else if (n <= 0.6) ch = 'b';
else if (n <= 0.9) ch = 't';
else {
  let r = random();
  if (r > 0.8) ch = '.';
  else ch = ' ';
}
```

The finesse bit

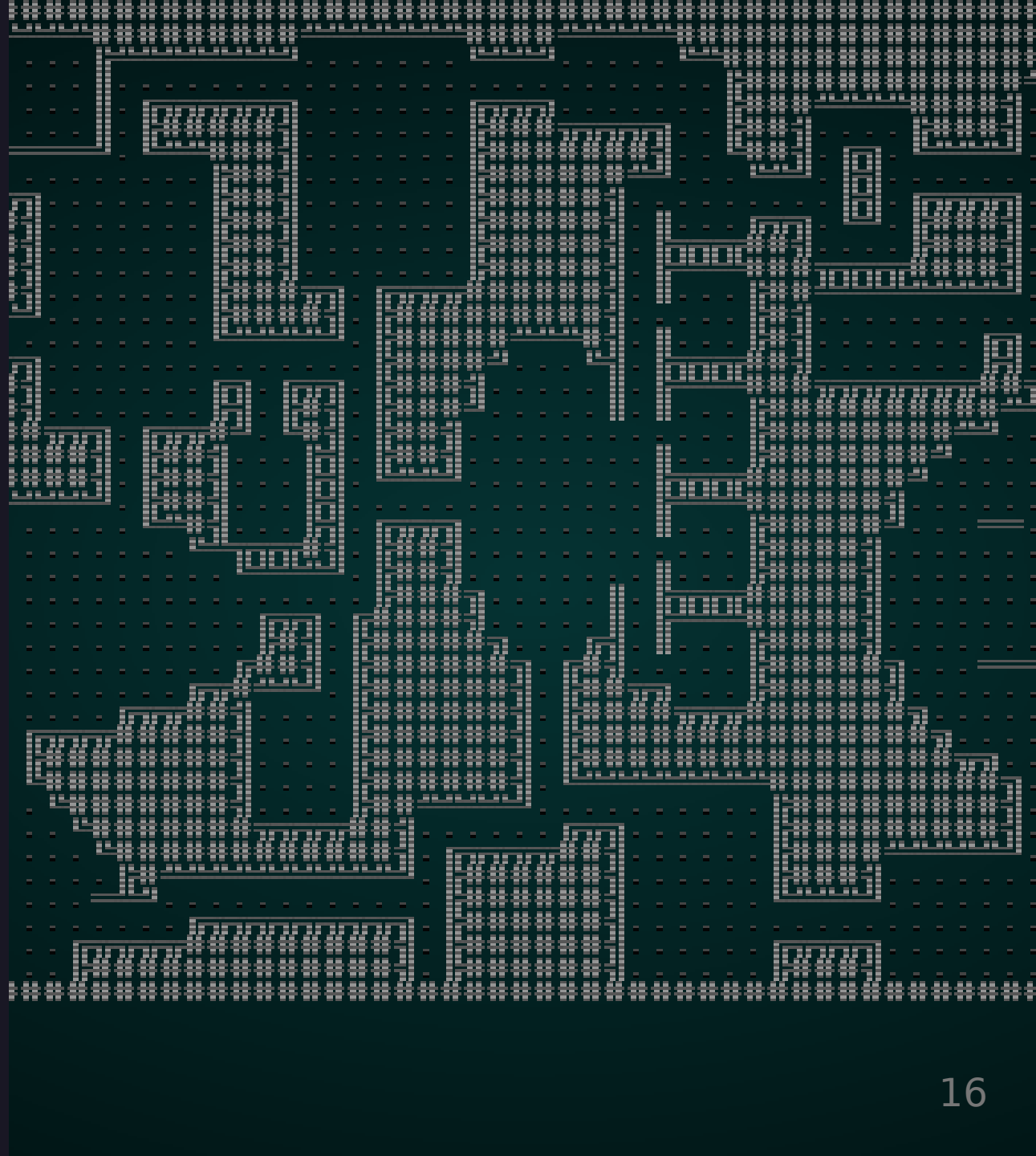
Now this is the tricky part - you need to play with the `zoom` value and the `noiseDetail` (at the top) values to get exactly the output you want!

Try varying the:

- `zoom` into the noise distribution (try `0.1`, `0.001`, etc.)
- https://p5js.org/reference/p5/noiseDetail/
  - The number of octaves (first parameter in `noiseDetail`) - `[1, 16]` usually show interesting values
  - The falloff amount (second parameter in `noiseDetail`) - `[0.0, 1.0]`

Other ways to do it!

Use a different algorithm!

- Rectangular room placement
- Cellular automata
- **Wave function collapse** ->
- ...

> **There's a cellular automata implementation in the demo - instead of building your map in `setup` try calling `game_map = CA();`**